Programming Assignment 1: Preferential Web Crawler for MedOverNet

Tibor Koderman, Jerneja Krajcar, Dino Džaferagić

I. Introduction

In this project, we built a standalone preferential crawler for the MedOverNet domain (https://medover.zurnal24.si/). The goal is to crawl relevant pages first, respect robots rules, avoid duplicate content, and store crawl data in PostgreSQL using the required crawldb schema.

The crawler is designed as a multi-worker system with a database-backed frontier. Each worker takes one URL, checks robots and politeness limits, downloads or renders the page, detects duplicates, stores the result, extracts new links and images, scores discovered URLs, and pushes them back to the frontier.

II. System architecture and implementation

A. High-level pipeline

The crawler processes each URL in the following order:

1) Claim next URL from the frontier.
2) If the canonical URL already exists in `crawldb.page` as `HTML/BINARY/DUPLICATE`, do not re-download; only insert the `link` record (from the source page to the already-known target).
3) Check robots rules and enforce politeness delay.
4) Download content (or render with headless browser when needed).
5) Detect exact duplicates using content hash.
6) Store page data and metadata in crawldb.
7) Parse outgoing links and images.
8) Score discovered URLs and enqueue them into frontier.

B. Main components

The implementation is split into clear modules:

- Worker orchestration and crawl loop: `scheduler.py`, `run_crawler.py`.
- Frontier and page persistence: `frontier_store.py`, `page_store.py`.
- Downloader and renderer: `downloader.py`, `renderer.py`.
- Parsing links and images: `parser.py`.
- Canonicalization and dedup utils: `url_canonicalizer.py`, `dedup.py`.
- Robots and politeness: `robots.py`, `politeness.py`.

C. Frontier queue semantics

The URL frontier is implemented in a dedicated table, `crawldb.frontier_queue`, rather than in `crawldb.page`. Each frontier row stores a canonical absolute `url`, `priority`, `source_url`, and `depth`, with a uniqueness constraint on `url`.

Enqueue uses UPSERT semantics and is concurrency-safe: if the URL already exists, priority is merged with `GREATEST`, depth/discovery metadata are kept from the best available values, and previously terminal rows (`COMPLETED`/`FAILED`) can be reactivated to `QUEUED`.

Workers claim URLs via an atomic transaction: candidates are read in priority order (`ORDER BY priority DESC, discovered_at ASC`) with `FOR UPDATE SKIP LOCKED`, then the selected row is moved to `LOCKED` and tagged with lease metadata (`locked_at`, `locked_by_worker_id`). Expired leases are automatically re-queued after a configurable TTL.

Queue state is enum-driven (`QUEUED`, `LOCKED`, `PROCESSING`, `COMPLETED`, `DUPLICATE`, `FAILED`). Completion reports map worker outcomes to terminal states, while retry outcomes map back to `QUEUED`. Politeness timing is enforced server-side through in-memory per-daemon/per-site cooldowns (minimum 5 s), not through a `frontier_next_fetch_time` column.

D. Division of work

Implemented in this part of the report draft:

- robots.txt support (`robots.py`),
- per-IP politeness/rate limiting (`politeness.py`),
- downloader (`downloader.py`),
- headless renderer (`renderer.py`),
- content-type classification and CLI test hooks (`main.py`, `config.py`).

Implemented by teammate (integrated architecture context):

- DB-backed frontier and worker loop orchestration,
- parser for links/images and URL normalization,
- relevance scoring and preferential URL ordering,
- exact hash dedup integration in persistence layer.

III. Preferential crawling strategy

The crawler uses a preferential strategy: every discovered URL is assigned a numeric priority (stored as `frontier_priority` in the frontier row), and workers always claim the currently due URL with the highest priority.

A. Scoring function

The current implementation uses a lightweight, configurable heuristic (`relevance.py`). For a discovered URL `u` with parent URL `p` at depth `d`, we compute:

- **Same-host boost:** if `host(u) == host(p)`, add `same_host_boost` (default: 10).
- **Allowed domain suffix boost (optional):** if the host matches any configured suffix, add `allowed_suffix_boost` (default: 20).
- **Keyword boosts (optional):** for each configured keyword occurring in the URL path/query, add `keyword_boost` (default: 5).
- **Depth penalty:** subtract `depth_penalty` $\cdot d$ (default: 0.2).

With the default configuration, the crawler primarily prefers same-host URLs and mildly prefers shallow depth. Domain-specific focusing is supported via configuration, but must be explicitly set.

B. Worked example with default parameters

For a child URL discovered on the same host at depth $d = 3$:

$$\text{score} = 10 - 0.2 \cdot 3 = 9.4$$

For an off-host URL at the same depth (no same-host boost):

$$\text{score} = -0.6$$

Therefore, same-host pages are explored first unless additional boosts are configured.

IV. Robots.txt and politeness

The crawler enforces robots and ethics constraints before each fetch:

- robots fetched per origin and cached,
- `Allow`/`Disallow` checked for crawler user-agent,
- `Crawl-delay` read when present,
- `Sitemap` URLs collected for optional frontier seeding,
- hard minimum delay of 5 seconds per target IP, even if robots does not define delay.

The implementation uses a thread-safe per-IP limiter. Effective delay is:

$$\text{effective_delay} = \max(5s, \text{robots_crawl_delay})$$

This design allows parallel workers while still respecting per-target pacing.

PLACEHOLDER (final report): insert one short robots compliance example from logs (allowed URL, disallowed URL, applied delay).

V. Downloader, renderer, and content-type handling

A. HTTP downloader

The downloader uses `requests` with crawler user-agent and timeout settings from configuration. It follows redirects and stores status code and final URL.

B. JS rendering

For pages where static HTML is not enough, the crawler can render using headless Chrome via Selenium. This mode is optional because it is slower, and we use it when needed for dynamic content.

C. Content-type policy

The crawler distinguishes HTML and binary resources:

- HTML: stored in `page.html_content`, `page_type = HTML`
- PDF: supported as optional binary download, otherwise metadata-only
- DOC/DOCX/PPT/PPTX: metadata listed in `page_data`, no binary payload required
- other non-HTML resources: marked as `BINARY`

In this assignment crawl HTML content by default, handle PDF where needed, and avoid storing unnecessary binary blobs.

VI. Deduplication

A. Exact deduplication

The crawler computes a content hash (SHA-256 of the exact HTML source) for HTML pages. The ingestion logic distinguishes:

- **Known canonical URL:** if the canonical URL already exists in `crawldb.page`, the crawler does not insert a new page row and does not treat this as a duplicate; it only records the hyperlink in `crawldb.link`.
- **Duplicate content:** if the canonical URL is new but the content hash matches an existing `HTML` page, a new row is inserted with `page_type_code = DUPLICATE`.

If the content hash already exists:

- new page is marked `DUPLICATE`,
- `duplicate_of` points to the original `HTML` page,
- duplicate page does not store full HTML content.

VII. Parameters and configuration

Crawler behavior is controlled by configuration variables. Main parameters include:

- number of workers,
- crawler user-agent (`fri-wier-{GROUP_NAME}`),
- minimum delay (must remain ≥ 5 s),
- robots timeout,
- download timeout,
- rendering timeout,
- optional PDF download flag.

The command-line entry point in `main.py` was used to test:

- robots checks and effective delay behavior,
- downloader behavior for HTML/PDF cases,
- renderer output size difference on dynamic pages.

PLACEHOLDER (final report): include final parameter table used for the 5000-page crawl run.

VIII. Problems and solutions

A. Robots and ethical crawling

Problem: parallel workers can violate politeness if delays are not shared.
Solution: server-side per IP locking and queue delagation, the shared queue holds timeouts for the next allowed fetch of each origin, when a worker requests a deque, and the server returns the highest priority URL that is not timed out, and adds it to the timeout list until the next allowed fetch time. This design allows multiple workers to operate in parallel while still respecting per-origin crawl delays. The new problem is that workers on different IPs are rate limited by the same delay, but this is a necessary tradeoff for ethical crawling. The other easy solution would just be to just do proxy switching, but this is not ethical.

B. Dynamic content

Problem: some links/content are injected by JavaScript.
Solution: optional Selenium rendering path for selected pages.

C. Frontier concurrency

Problem: multiple workers may claim the same URL.
Solution: The workers don't connect to the database directly, this is also a security design to allow workers to run on seperate machines without exposing the database to the networ. Instead the frontier is implemented on the server. When a worker is in a idle state, the server deques the highest priority URL, and locks it by updating it's status in the database, this removes it from the frontier candidates. To ensure that the queue is fast and responsive, the server maintains an in-memory queue of the urls in the fronntier. The databae table is also indexed by the frontier_priority and frontier_next_fetch_time, so that the server can quickly find the next available URL to assign to a worker. If a worker fails to fetch a URL, the server can re-enqueue it by updating its status and frontier_next_fetch_time in the database.

D. Duplicate handling

Problem: same content may appear under different URLs.
Solution: exact hash check and `DUPLICATE` page type with pointer to original.

E. Spider traps and loops

Problem: repeated navigation patterns and low-value loops can waste crawl budget.
Solution: canonical URL uniqueness in DB, relevance scoring, and robots compliance constraints.

IX. Results, statistics, and sisualization (Placeholders)

This section is intentionally a scaffold for the final submission stage.

A. Crawldb statistics

PLACEHOLDER: fill values from SQL queries after final crawl run.

Table I
Main crawl statistics (seed domain and full crawldb)

Metric	Seed domain (MedOverNet)	Full crawldb
Number of sites	[TBD]	[TBD]
Number of HTML pages	[TBD]	[TBD]
Number of duplicates	[TBD]	[TBD]
Number of binary pages	[TBD]	[TBD]
Number of images	[TBD]	[TBD]
Average images per HTML page	[TBD]	[TBD]

Table II
Binary documents by type

Data type	Count
PDF	[TBD]
DOC	[TBD]
DOCX	[TBD]
PPT	[TBD]
PPTX	[TBD]

B. Link graph visualization

PLACEHOLDER: add graph visualization figure generated in Gephi/D3/vis.js.

PLACEHOLDER: Link graph figure (nodes=pages, edges=links)

Figure 1. Crawl link structure (placeholder).

C. Example pages/images

PLACEHOLDER: add 1–2 representative screenshots or extracted image examples.

X. Conclusion

We prepared a working foundation for a preferential MedOverNet crawler with core requirements: robots compliance, politeness control, HTML/binary handling, optional JS rendering, exact hash dedup support, and a multi-worker DB-backed crawl design. The preferential strategy is implemented via a configurable scoring function that prioritizes same-host and shallow URLs by default.